SWEN 221

Marco Servetto
CO258
www.youtube.com/MarcoServetto


Finally, streams!

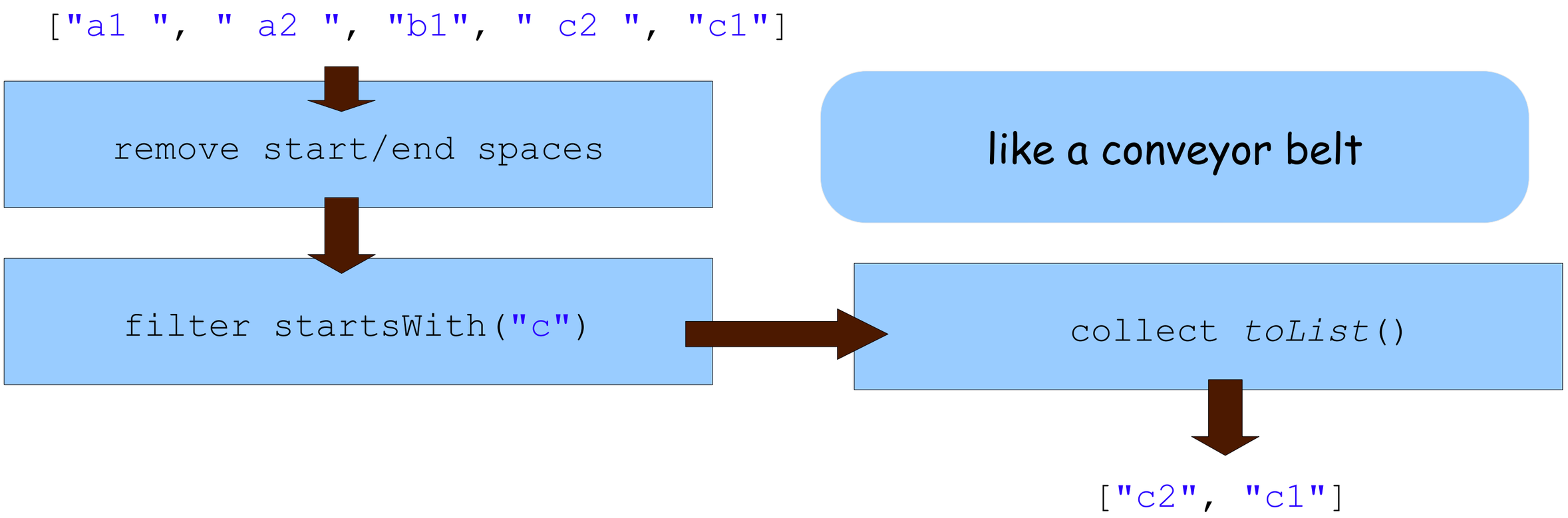
Java: Streams and collections

(Not correlated with InputStreams etc..)

Java: Streams and collections

(Not correlated with InputStreams etc..)

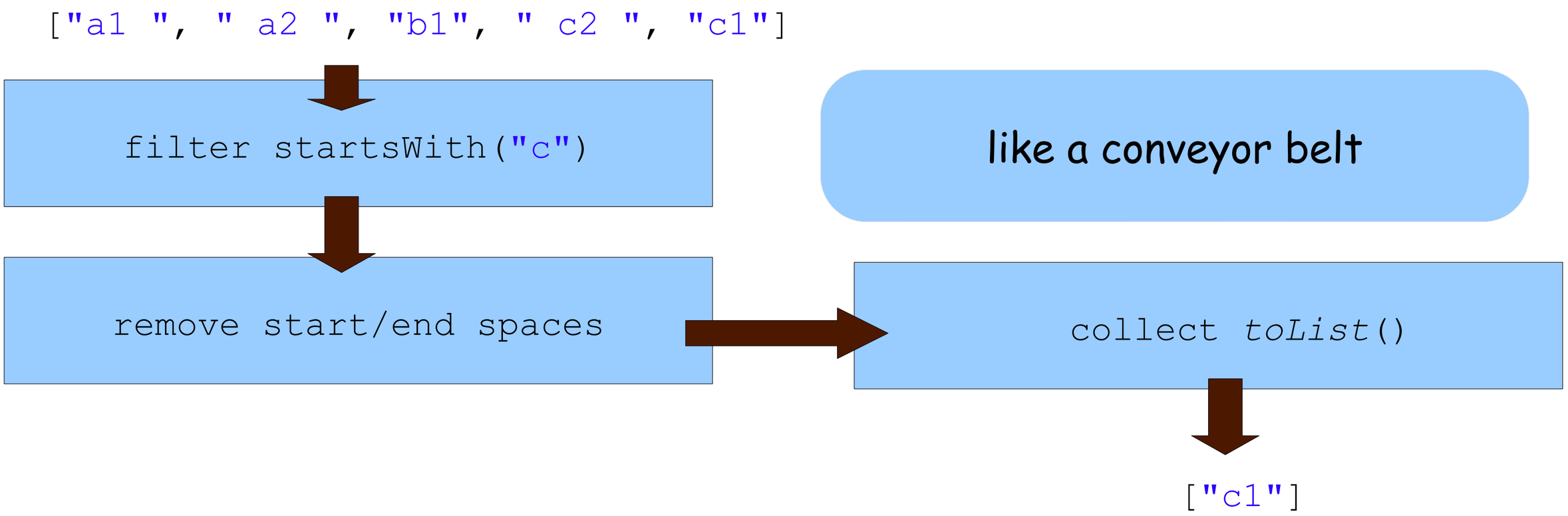
- Stream: Rich library defining sort of a sub-language to query and process collections.



Java: Streams and collections

(Not correlated with InputStreams etc..)

- Order of operations is important!



Java: Streams and collections

```
//list from strings
List<String> myList= List.of("a1 ", " a2 ", "b1", " c2 ", "c1");
myList = myList.stream()//our entry point
  .filter(s -> s.startsWith("c"))//select only some stuff
  .map(s->s.trim())//mapping
  .toList();//myList == [c1]
```

- .stream is our **entry point** for this rabbit-hole
- .filter and .map are **intermediate operations**:
  streams in, streams out!
- .toList is a **terminal operation**; in this case it produces a list.

Java: Streams and collections

- Until the terminal operation is called, streams are just accumulating instructions, without executing them.

```
List<String>   ps= List.of("Bob");
Stream<String>  s= ps.stream()
  .map(p->{ System.out.print(p); return p; });
System.out.print(" The name is ");
ps = s.toList();
```

- What is printed here?
  - (A) The name is Bob
  - (B) Bob The name is

Map and filter

```
List<Person> persons= List.of(...);
List<Student> readyForSWEN225= persons.stream()
  .filter(p-> p instanceof Student)
  .map(p-> (Student)p)
  .filter(s-> s.marks.containsKey("SWEN221"))
  .toList();
```

Map and filter, equivalent for loop

```
List<Person> persons= List.of(...);                    //a
List<Student> readyForSWEN225= new ArrayList<>();      //b
for(Person p: persons){                                //c
  if (!(p instanceof Student)){ continue; }            //d
  Student s = (Student)p;                              //e
  if (!s.marks.containsKey("SWEN221")){ continue; }//f
  readyForSWEN225.add(s);                              //g
}
```

```
List<Person> persons= List.of(...);                    //a
List<Student> readyForSWEN225= persons.stream()        //b+c
  .filter(p-> p instanceof Student)                    //d
  .map(p-> (Student)p)                                 //e
  .filter(s-> s.marks.containsKey("SWEN221"))          //f
  .toList();                                           //g
```

Map, filter, reduce

```
List<Person> persons= List.of(...);
List<Student> readyForSWEN225= persons.stream()
   .filter(p-> p instanceof Student)
   .map(p-> (Student)p)
   .filter(s-> s.marks.containsKey("SWEN221"))
   .toList();
```

Here we collect in a list

```
List<Person> persons= List.of(...);
Optional<Student> younger= persons.stream()
   .filter(p-> p instanceof Student)
   .map(p-> (Student)p)
   .filter(s-> s.marks.containsKey("SWEN221"))
   .reduce((s1,s2)->{
      if (s1.age < s2.age){ return s1; }
      return s2;
      });
```

Here we take the younger one

Reduce

```
//Optional if you just accumulate
Optional<Student> younger= ...
  .reduce((s1,s2)->{...});
```

Reduce

```
//Optional if you just accumulate
Optional<Student> younger= ...
  .reduce((s1,s2)->{...});

//Sure if you provide a starting point
Student alice      = ...
Person atLeastAlice= ...
  .reduce(alice,(s1,s2)->{...});
```

Stream exercise: Genetic algorithm

- Compute fitness for candidate solutions and select the best ones.

- For example, you may have a list of paper aeroplanes, and you want to record the best ones!

Stream exercise: Genetic algorithm

You can easily do that using streams:

```
List<Aeroplane> attempts= ...

attempts.stream()//first, cache the fitness
  .forEach( a-> a.computeAverageFlightTime() );

List<Aeroplane> best20= attempts.stream()
  .sorted( (a1,a2)-> a1.getFlightTime() - a2.getFlightTime() )
  .limit(20)//take the first 20
  .toList();
```

sorted + limit

- Currently, this is going to sort the whole list and then select the best 20.
  Not the best performance, but pretty close if the list is under 10 million elements.
- New versions of streams could make it even faster.

Stream exercise: Genetic algorithm

You can easily do that using streams:

```
List<Aeroplane> attempts= ...

attempts.stream()//first, cache the fitness
  .forEach( a-> a.computeAverageFlightTime() );

List<Aeroplane> best20= attempts.stream()
  .sorted( (a1,a2)-> a1.getFlightTime() - a2.getFlightTime() )
  .limit(20)//take the first 20
  .toList();
```

`computeAverageFlightTime` can be slow,

- Must be done for all your attempts!

- Modern hardware have multiple processors.

Stream exercise: Genetic algorithm

In real life, you could try to fly aeroplanes using multiple rooms at the same time,

- To test 100 aeroplanes, for 1 minutes each, it would take 100 minutes.
- Having 10 friends and 10 testing chambers, you can run parallel tests, to finish in 10 minutes

- The same idea applies with multiple processors:

  you may run 8 `computeAverageFlightTime`

  at the same time using 8 cores.

Parallel Stream

- Streams can use multiple processors.
  No need to use threading explicitly.

```
attempts.stream()//sequential
  .forEach( a-> a.computeAverageFlightTime() );
List<Aeroplane> best20= attempts.stream()
  .sorted( (a1,a2)-> a1.getFlightTime() - a2.getFlightTime() )
  .limit(20)
  .toList();
```

As before

Parallel Stream

- Streams can use multiple processors.
  No need to use threading explicitly.

As before

```
attempts.stream()//sequential
  .forEach( a-> a.computeAverageFlightTime() );
List<Aeroplane> best20= attempts.stream()
  .sorted( (a1,a2)-> a1.getFlightTime() - a2.getFlightTime() )
  .limit(20)
  .toList();
```

In parallel

```
attempts.parallelStream()//parallel
  .forEach( a-> a.computeAverageFlightTime() );
List<Aeroplane> best20= attempts.parallelStream()
  .sorted( (a1,a2)-> a1.getFlightTime() - a2.getFlightTime() )
  .limit(20)
  .toList();
```

Parallel Stream

- Just like that?
  Just call .parallelStream() instead of .stream()

  No need to use threading explicitly?
- Yep!
- And by the way, in any application, if you have to use treads explicitly, you are doing something that is at research level, you are expanding what humans believe possible.
- So either you are doing it **wrong**, or be sure to be paid accordingly.

Parallel Stream and reduce

- A third form of reduce is useful when using parallel streams. Takes 3 parameters
  - an initial value (int in this case).
  - a way to compose a (int) value with a stream element (Person) to produce a new (int) value.

Parallel Stream and reduce

- A third form of reduce is useful when using parallel streams. Takes 3 parameters
  - an initial value (int in this case).
  - a way to compose a (int) value with a stream element (Person) to produce a new (int) value.
  - a way to compose two (int) values into a new one.

Parallel Stream and reduce

- A third form of reduce is useful when using parallel streams. Takes 3 parameters
  - an initial value (int in this case).
  - a way to compose a (int) value with a stream element (Person) to produce a new (int) value.
  - a way to compose two (int) values into a new one.
- parallelStream divides your work in jobs and put them together.

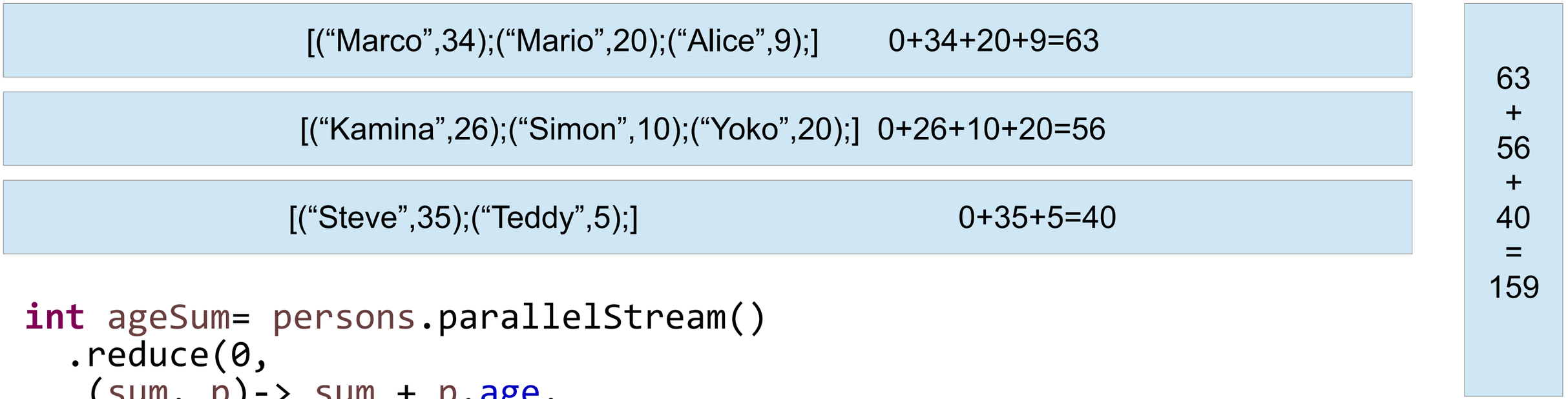
Parallel Stream and reduce

- A third form of reduce is useful when using parallel streams. Takes 3 parameters
  - an initial value (int in this case).
  - a way to compose a (int) value with a stream element (Person) to produce a new (int) value.
  - a way to compose two (int) values into a new one.
- parallelStream divides your work in jobs and put them together.

```
int ageSum= persons.parallelStream()
  .reduce(0,
   (sum, p)-> sum + p.age,
   (sum1, sum2)-> sum1 + sum2);
```

Parallel Stream and reduce

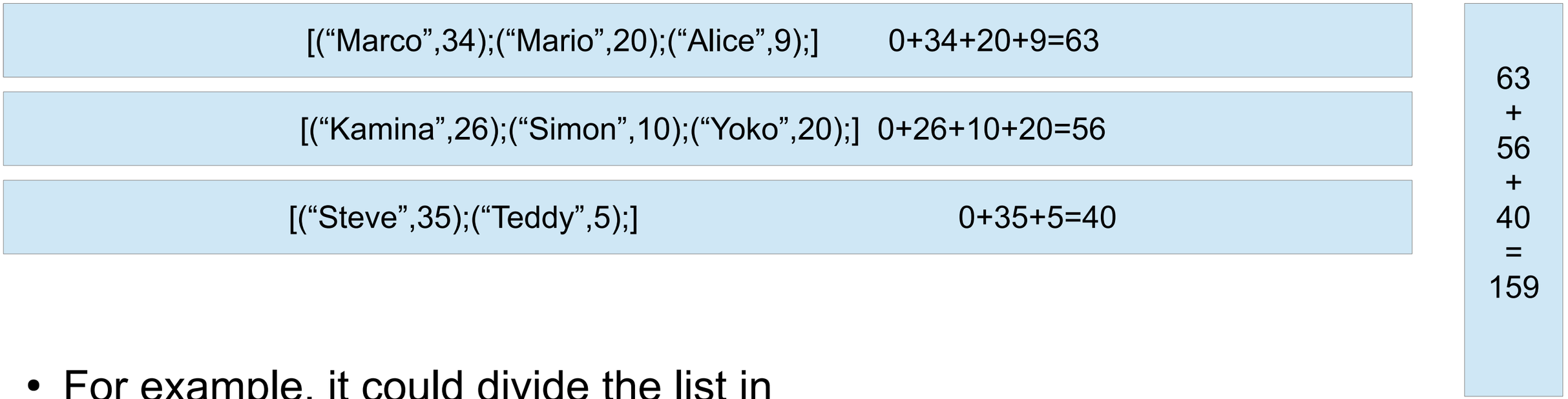
- parallelStream figures out how to divide your work in jobs and how to put them together.




```
int ageSum= persons.parallelStream()
  .reduce(0,
   (sum, p)-> sum + p.age,
   (sum1, sum2)-> sum1 + sum2);
```

Parallel Stream and reduce

- parallelStream figures out how to divide your work in jobs and how to put them together.



- For example, it could divide the list in 8 sublist of approximately the same length, compute the sum of the sublists and then compute the grand total.

More parallel streams

- More Java parallel programming and more parallel streams in NWEN303

More on Streams

- Presented by CAT

  Computerized
  Automated
  Tutor

Hello Class !
Nice to mewt you.

I'm CAT
I will halp you thinking
with streams

Streams are great to work with lists/collections

The main thing to keep in mind is the SIZE

The input size and the output size

```
List<Person> persons= List.of(...);
List<String> names  = persons.stream()
   .map( p -> p.name )
   .toList();
```

Are there the same? Use map!

--.map --keeps the same size. One element flows into one element

```
List<Person> persons= List.of(...);
List<Person> nices  = persons.stream()
   .filter( p -> iCanHasCheezburgerFrom(p) )
   .toList();
```

Are there elements you no like?

--.filter -- them away. If they do not pass your judgment, they are out

```
persons.stream()
   .flatMap( p -> p.friends().stream() )
   .forEach( p -> cat.stealCheezburger(p) );

//forEach does an action for all the elements
//Here for all the friends of all the persons
```

You want MORE elements ?

--.flatMap -- is your friend. Just specify a stream of elements to fit in place of a single element.

```
Optional<Person> owner= persons.stream()
   .max((p1,p2)->
     Double.compare(foodOf(p1), foodOf(p2)) );
```

You want to get a single result?

Focus on terminal operations. But be careful. This will push you out of streamy-land

```
//Two different ways to get the same result
 Optional<Person> owner= persons.stream()
    .max((p1,p2)->
      Double.compare(foodOf(p1), foodOf(p2)) );

 Optional<Person> owner= persons.stream()
    .max( Comparator.comparing(p->foodOf(p)) );
//Many funky API pieces to combine!
//Memorization is needed here...
```

streams can always be empty (like my belly!)

What is the max of an empty stream? There is no max!! That is why max/min/reduce and others gives us an Optional result!

```java
var home= persons.stream()
   .anyMatch(p->atHome(p));
if(home){/*..*/}

assert persons.stream()
   .anyMatch(p->atHome(p));
//Great for asserts!
```

You want to take decisions?

Terminal operations again! Look for --.anyMatch-- and his its friends!

```
Optional<Person> friend= persons.stream()
    .filter( p->atHome(p) )
    .findFirst();
//Optimized: filters until it finds the first
//and then stops exploring the list
```

Someone is at home.. but who?

To find elements use --.filter-- and --.findFirst--

```
List<String> names   = List.of(...);
List<String> surnames= List.of(...);
assert names.size() == surnames.size();
List<Food> yummy     = IntStream.range(0, names.size())
   .mapToObj(i-> new Person(names.get(i), surnames.get(i)))
   .map( p0-> coerceFoodFrom(p) )
   .toList();
//'Zipping' two collections. Just use the indexes
//Long stream chains are GOOD.
//Gives the library many optimization options!
```

Down to the indexes!

--.range-- is your way out from many sticky situations!

```
List<String> names   = List.of(...);
List<String> surnames= List.of(...);
assert names.size() == surnames.size();
List<Food> yummy     = IntStream.range(0, names.size())
  //IntStream ~= Stream<Integer>
  .mapToObj(i-> new Person(names.get(i), surnames.get(i)))
  //Stream<Person>
  .map( p0-> coercingFoodFrom(p) )
  //Stream<Food>
  .toList();
```

Follow the types!

they will guide you like bread crumbs

```
.stream  //list or set to stream
IntStream.range //to work on indexes. Very flexible.
.map     //1 to 1 transformations
.flatMap //1 to 0..n transformations
.filter  //take stuff out of the pipeline
.filter+findFirst //select an element by a property
.forEach          //do an action on all elements
.max/.min/.reduce //tons of terminal operations to study
.anyMatch/.allMatch/.noneMatch //test for properties
```

Summary!

Learn Stream-fu!

More CAT on
https://www.youtube.com/watch?v=ieEiQW2V6-E

Next:
Making our own abstractions